1 Введение

В организациях любого масштаба значительная часть операционной деятельности связана с обработкой документов. Договоры, счета, приказы, служебные записки и отчеты поступают из различных источников, хранятся в разных форматах и проходят через множество участников бизнес-процесса. На практике это приводит к накоплению смешанных архивов, в которых документы необходимо быстро идентифицировать и разложить по типам.

Особую сложность представляет одновременная работа с несколькими представлениями одного и того же документа. В информационных системах могут присутствовать как текстовые версии файлов, так и сканы, фотографии, выгрузки из почты и сжатые архивы, передаваемые между подразделениями. Следовательно, прикладное программное решение должно уметь работать не только с чистым текстом, но и с визуальным макетом документа, а также с пакетной обработкой массивов файлов.

В условиях цифровизации документооборота возрастает потребность в компактных сервисах, которые можно быстро развернуть локально и встроить в существующий рабочий процесс. Такой сервис должен не только классифицировать отдельный документ, но и решать практическую задачу первичной маршрутизации входящего массива файлов: определить тип документа, сохранить историю обработки и сформировать аккуратно отсортированный результат для дальнейшей работы.

Актуальность темы обусловлена сочетанием двух факторов. С одной стороны, машинное обучение предоставляет инструменты для классификации текста и изображений без ручного кодирования большого числа правил. С другой стороны, low-code средства позволяют быстро реализовать web-интерфейс и превратить исследовательский прототип в реально используемое прикладное приложение.

Целью выпускной квалификационной работы является разработка программной системы интеллектуальной классификации и пакетной сортировки документов на языке Python с low-code интерфейсом, REST API и локальной базой данных.

Для достижения поставленной цели необходимо решить следующие задачи: проанализировать предметную область автоматизации документооборота; определить прикладной сценарий пакетной обработки документов; спроектировать архитектуру программной системы, структуру базы данных и пользовательские сценарии; реализовать модули классификации текста, сканов и ZIP-архивов; разработать интерфейс взаимодействия пользователя с системой; выполнить тестирование и оценить результаты работы приложения.

Объектом исследования являются процессы автоматической классификации и маршрутизации электронных документов. Предметом исследования выступают алгоритмы и программные средства, обеспечивающие распознавание типа документа по текстовому содержимому, по визуальному макету и по составу пакетного архива.

Практическая значимость работы заключается в создании локального программного продукта, пригодного для использования в роли демонстрационного и учебного стенда, а также в качестве прототипа сервиса первичной сортировки корпоративных документов. Система не требует отдельного сервера баз данных, запускается в среде PyCharm и воспроизводит полный инженерный контур: данные, модели, интерфейс, тесты, отчеты и документацию.

Структура выпускной квалификационной работы включает введение, четыре главы основной части, заключение, список использованных источников и приложение. В первой главе рассматриваются предметная область, актуальность, цель, задачи и существующие аналоги. Во второй главе описываются архитектура разрабатываемого ПО, структура БД и диаграммы

сценариев. Третья глава посвящена этапам реализации и поэтапному описанию блоков приложения. В четвертой главе приводятся результаты тестирования, преимущества и ограничения решения.

2 ГЛАВА 1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ПРАКТИЧЕСКОЙ ЗАДАЧИ

2.1 Предметная область автоматизации документооборота

Предметной областью работы является первичная обработка входящих документов внутри организации. Под такой обработкой понимается определение типа документа, выбор дальнейшего маршрута и подготовка документов к последующей регистрации, согласованию или хранению.

В реальном документообороте источники данных неоднородны. Пользователь может получить текстовое описание документа из электронной почты, скан из МФУ или фотографию с мобильного устройства, а также архив, в котором смешаны несколько видов файлов. Следовательно, прикладной сервис должен учитывать как текстовые, так и визуальные признаки документов.

Для решения задачи в рамках ВКР выбран ограниченный, но практически значимый набор классов: договор, счет, приказ, служебная записка и отчет. Данные типы часто встречаются в административной, договорной и отчетной деятельности организаций, а также хорошо различаются по содержанию и по структуре макета.

2.2 Актуальность разработки

Ручная сортировка большого массива документов создает издержки времени и повышает риск ошибок. Сотруднику необходимо открыть файл, определить его назначение, выбрать папку хранения и иногда дополнительно проверить наличие обязательных реквизитов. При поступлении смешанного архива с десятками файлов такая работа становится монотонной и плохо масштабируется.

Дополнительную проблему создает необходимость быстрой предварительной фильтрации до внедрения полноценной ECM-системы. Во многих учебных и пилотных проектах нужен легкий локальный инструмент, который показывает, что маршрутизация документов может быть

автоматизирована даже без тяжелой инфраструктуры и дорогостоящего внедрения.

С научной точки зрения интерес представляет объединение нескольких модальностей в едином приложении. Текстовая модель опирается на лексические признаки, визуальная модель использует структуру документа, а пакетный режим демонстрирует перенос результатов классификации на практический сценарий работы с архивом.

2.3 Цель и задачи проектирования

Практическая постановка задачи в данной работе формулируется следующим образом: необходимо разработать локальное приложение, которое сможет автоматически отнести отдельный документ или целый архив документов к одному из заданных типов и сформировать понятный результат для пользователя.

В отличие от абстрактной демонстрации мультимодальной классификации, выбранный сценарий имеет прямое прикладное назначение. Пользователь получает не только прогноз модели, но и инструмент первичной расфасовки документов по категориям, что упрощает последующую передачу файлов в систему электронного документооборота.

Таким образом, программный продукт должен решать одновременно три класса задач: одиночную классификацию текста, одиночную классификацию скана документа и пакетную обработку архива. Каждому из этих сценариев необходимо обеспечить журналирование, удобный интерфейс и воспроизводимость результатов.

2.4 Характерные признаки типов документов

Договор обычно содержит формализованную вводную часть со сторонами соглашения, предметом договора, порядком расчетов, условиями ответственности и блоком подписей. Для данного класса важны термины, связанные с обязательствами сторон, сроками и приложениями, а в визуальном

представлении часто встречаются симметрично расположенные подписи и плотные абзацы текста.

Счет отличается наличием реквизитов поставщика, табличной части с номенклатурой и суммами, итогового блока оплаты и реквизитов для перечисления денежных средств. Для текстовой модели характерны слова 'итого', 'реквизиты', 'НДС', 'сумма к оплате', а для визуальной модели - крупная таблица и компактный блок итогов.

Приказ, служебная записка и отчет имеют разные коммуникативные функции. Приказ оформляет управленческое распоряжение, служебная записка служит внутренним обращением или обоснованием, а отчет систематизирует результаты деятельности и часто содержит таблицы или диаграммы. Наличие такого различия позволяет построить обучающие выборки с выразительными текстовыми и визуальными признаками.

2.5 Риски и последствия ошибочной маршрутизации документов

Ошибочная маршрутизация документов приводит не только к потерям времени, но и к искажению управленческих процессов. Если счет попадает в договорной контур, а приказ ошибочно помещается в архив отчетов, дальнейшая обработка документа становится затрудненной и требует ручного поиска причины ошибки.

На уровне делопроизводства подобные ошибки означают рост нагрузки на сотрудников, появление повторных операций и снижение доверия к цифровым инструментам. Именно поэтому даже система предварительной сортировки должна быть не просто демонстрационной, а устойчивой и объяснимой для конечного пользователя.

Разрабатываемое приложение решает эту проблему путем сочетания трех механизмов: автоматической классификации, прозрачного показа уверенности модели и ведения истории обработки. Такой набор позволяет не только ускорить

работу, но и отследить, почему конкретный документ оказался в той или иной категории.

2.6 Анализ существующих решений

На рынке корпоративных информационных систем уже существуют зрелые решения, обеспечивающие электронный документооборот, маршрутизацию, хранение и согласование документов. Однако такие платформы обычно ориентированы на комплексное внедрение и требуют существенной организационной подготовки.

Для сравнения с разрабатываемым ПО целесообразно рассмотреть ABBYY FlexiCapture, Directum RX, ELMA365 ECM и 1С:Документооборот. Указанные системы охватывают близкую прикладную область, но решают ее на уровне enterprise-платформ с широким набором дополнительных функций.

Сравнение аналогов показывает, что нишей разрабатываемого продукта является компактный воспроизводимый сервис, в котором акцент сделан на задаче классификации и пакетной сортировки документов, а не на полном жизненном цикле ECM-системы.

Таблица 1.1 - Сравнение аналогов и разрабатываемого решения

Решение	**Основной функционал**	**Сильные стороны**	**Ограничения**
ABBYY FlexiCapture	Интеллектуальный захват, OCR, классификация и экспорт документов	Высокая зрелость, enterprise-функции, интеграции	Избыточность и высокая стоимость для локального демонстрационного стенда
Directum RX	Корпоративный документооборот, маршрутизация, работа с договорами и ОРД	Глубокая поддержка бизнес-процессов и контроля исполнения	Ориентирован на крупное внедрение, а не на компактный исследовательский прототип

ELMA365 ECM	Хранение, регистрация, согласование, маршруты и электронный архив	Гибкая low-code настройка маршрутов и карточек документов	Сильная зависимость от экосистемы платформы и прикладных модулей
1С:Документооборот	Работа с корреспонденцией, приказами, договорами и маршрутами согласования	Широкая распространенность и интеграция с учетными системами 1С	Сфокусирован на полноценной ECM-системе, а не на задаче ML-классификации архивов
Разрабатываемое ПО	Локальная классификация текста, сканов и ZIP-архивов документов	Прозрачная архитектура, воспроизводимость, простая демонстрация ML-контура	Синтетический набор данных и ограниченный перечень типов документов

2.7 Критерии эффективности разрабатываемой системы

Эффективность разрабатываемой системы в рамках ВКР оценивается по нескольким критериям. Первый критерий - корректность отнесения документа к целевому классу. Второй критерий - скорость ответа системы в интерактивном режиме. Третий критерий - способность системы без ручного вмешательства сформировать полезный итоговый архив.

Наряду с этим важны критерии инженерной зрелости: переносимость на другую машину, наличие тестов, автоматическая пересборка артефактов и понятная структура проекта. Такие свойства особенно значимы для ВКР, поскольку работа должна быть не только исследовательски корректной, но и технически воспроизводимой.

Следовательно, итоговая оценка решения не ограничивается лишь точностью модели. Полезность приложения определяется тем, насколько полно оно закрывает реальный пользовательский сценарий и насколько надежно переиспользуется вне исходной среды разработки.

Таблица 1.2 - Сопоставление признаков целевых классов документов

Класс	**Текстовые признаки**	**Визуальные признаки**	**Функция документа**
Договор	Стороны, предмет договора, условия, ответственность	Плотные абзацы, парные подписи	Фиксация соглашения между сторонами
Счет	Реквизиты, сумма, НДС, оплата	Крупная таблица и блок итогов	Основание для оплаты
Приказ	Распорядительные формулировки, пункты поручений	Заголовок и нумерованные блоки	Управленческое распоряжение
Служебная записка	Адресат, просьба, обоснование	Краткое тело, реквизиты адресата	Внутреннее служебное обращение
Отчет	Показатели, выводы, рекомендации	Таблицы, диаграммы, аналитические блоки	Подведение итогов работы

2.8 Практическая и научная ценность работы

Практическая ценность работы состоит в реализации законченного приложения, которое можно использовать как локальный инструмент предварительной маршрутизации документов и как демонстрационный стенд для защиты ВКР. Пользователь получает готовый интерфейс, пакетный режим и набор воспроизводимых артефактов.

Научная ценность выражается в сравнении двух разных способов распознавания типа документа в единой системе: по содержимому текста и по структуре визуального макета. Дополнительно исследование показывает, как результаты классификации могут быть перенесены в прикладной сценарий сортировки массива документов без ручного вмешательства.

2.9 Выводы по первой главе

В первой главе определена предметная область автоматизации входящего документооборота и обоснована актуальность создания компактного сервиса для классификации и сортировки документов. Показано, что прикладная задача может быть сформулирована как автоматическое распределение отдельных файлов и архивов по типам документов.

Проведенный анализ аналогов подтвердил, что существующие корпоративные платформы решают широкий круг задач, но для учебного и исследовательского проекта целесообразно создать самостоятельное локальное приложение с четко ограниченным сценарием, прозрачной архитектурой и воспроизводимой экспериментальной базой.

3 ГЛАВА 2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ДЛЯ КЛАССИФИКАЦИИ И СОРТИРОВКИ ДОКУМЕНТОВ

3.1 Описание разрабатываемого программного обеспечения

Разрабатываемое программное обеспечение представляет собой локальное web-приложение на языке Python. Система объединяет серверную часть, low-code интерфейс, две модели машинного обучения, локальную базу данных и набор генераторов отчетных материалов. Такой состав позволяет воспроизвести полный жизненный цикл прикладного ML-сервиса в рамках одного проекта.

В системе поддерживаются три прикладных сценария. Первый сценарий связан с классификацией текста документа. Второй предназначен для работы со сканом или изображением документа. Третий сценарий реализует пакетную обработку ZIP-архива и наиболее полно раскрывает практическую полезность приложения, поскольку автоматизирует расфасовку смешанного массива файлов по типам документов.

Результатом работы приложения является не только прогноз по одному документу, но и набор служебных объектов: записи в истории, реестр моделей, CSV-сводка пакетной обработки и новый архив, в котором файлы разложены по подкаталогам, соответствующим распознанным типам документов.

Таблица 2.1 - Функциональные требования к системе

Код	**Требование**	**Содержание**
F1	Классификация текста документа	Пользователь вставляет текст и получает тип документа и вероятности
F2	Классификация скана документа	Пользователь загружает изображение и получает прогноз по макету документа
F3	Пакетная сортировка архива	ZIP-архив автоматически разбирается по категориям документов

F4	Формирование выходного архива	Система возвращает ZIP с разложением по папкам и CSV-сводкой
F5	Хранение истории	Сохраняются одиночные запросы, пакетные прогоны и сведения о моделях
F6	Отображение метрик	Пользователь видит качество моделей и последние результаты обработки

3.2 Нефункциональные требования и ограничения

Помимо функциональности система должна удовлетворять ряду нефункциональных требований. Для учебного и исследовательского проекта особенно важны переносимость, отсутствие тяжелой инфраструктуры, наглядность результатов и возможность быстрой повторной проверки на другой машине.

Требование переносимости связано с фактом, что приложение демонстрируется в рамках ВКР и должно открываться в типичной среде разработки без отдельной настройки сервера БД, облачной инфраструктуры или внешних хранилищ. Именно поэтому выбран локальный стек с SQLite и файловыми артефактами внутри проекта.

Требование безопасности проявляется в пакетном режиме: архив нельзя извлекать без проверки путей, иначе появляется риск записи файлов за пределы рабочего каталога. В реализации предусмотрена валидация путей и контроль набора поддерживаемых расширений.

Таблица 2.2 - Нефункциональные требования к системе

Код	**Требование**	**Содержание**
NF1	Переносимость	Проект должен запускаться на локальной машине без

		внешней СУБД и отдельных сервисов
NF2	Производительность	Интерактивные сценарии должны завершаться за доли секунды после прогрева
NF3	Воспроизводимость	Модели, графики и документы должны пересобираться из одного репозитория
NF4	Прозрачность результатов	Пользователь должен видеть историю запросов, метрики и сформированный архив
NF5	Безопасность файловой обработки	ZIP-архивы должны извлекаться с проверкой путей и поддерживаемых форматов

3.3 Информационные потоки в системе

Система оперирует несколькими типами информационных потоков. Первый поток связан с пользовательским вводом: текстом, изображением или ZIP-архивом. Второй поток состоит из внутренних служебных данных - прогнозов моделей, времени обработки, метрик и путей к артефактам. Третий поток представлен выходными материалами, такими как CSV-сводка и итоговый архив с разложением по типам документов.

Разделение потоков важно для проектирования модулей. Пользовательский поток должен быть быстро обработан и безопасно валидирован. Служебный поток должен фиксироваться в БД и быть пригодным для анализа. Выходной поток должен быть ориентирован на дальнейшую эксплуатацию пользователем или внешней системой.

Фактически проектируемое ПО связывает пользовательский уровень, алгоритмический уровень и уровень хранения. Именно эта связность обеспечивает приложению практический смысл: классификация сразу

преобразуется в полезный операционный результат, а не остается изолированным числовым ответом модели.

3.4 Диаграмма прецедентов

Для формализации пользовательских сценариев была построена диаграмма прецедентов. Она фиксирует, какие действия должен поддерживать интерфейс системы и какие результаты ожидаются после выполнения этих действий.

С точки зрения практического использования наиболее важным прецедентом является пакетная сортировка архива, поскольку именно он связывает исследовательскую задачу классификации с реальным прикладным эффектом для делопроизводства: быстрым разбором смешанного входящего массива документов.

Рисунок 2.1 - Диаграмма прецедентов системы

3.5 Архитектура программной системы

Архитектура приложения построена по модульному принципу. Пользователь взаимодействует с системой через low-code web-интерфейс Gradio, который передает запросы в REST API на базе FastAPI. Далее вызовы попадают

в сервисный слой, где сосредоточена бизнес-логика классификации, журналирования и пакетной сортировки.

Внутри сервисного слоя используются два независимых алгоритмических контура. Первый контур отвечает за классификацию текста документа по TF-IDF признакам и логистической регрессии. Второй контур работает с визуальным макетом документа и определяет тип файла по структуре скана с помощью Random Forest.

Отдельным компонентом выделен архивный модуль. Он отвечает за безопасное извлечение файлов из ZIP, определение модальности, запуск нужной модели и формирование выходного архива. Это делает архитектуру расширяемой: при необходимости к существующим классам документов можно добавить новые правила маршрутизации и дополнительные алгоритмы обработки.

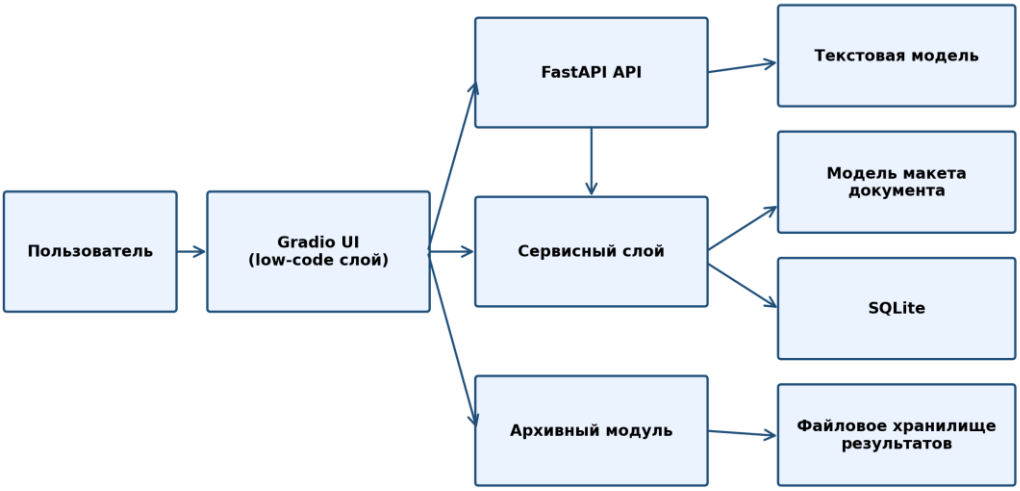

Рисунок 2.2 - Архитектура разрабатываемой системы

3.6 Состояния документа в рамках сценария обработки

С точки зрения проектирования полезно рассматривать документ как объект, проходящий через несколько состояний. На первом шаге документ

находится в необработанном виде: как текст, скан или файл внутри архива. Затем он переходит в состояние подготовленного входа, когда для него определена модальность и выбран алгоритмический маршрут.

После выполнения модели документ получает состояние классифицированного объекта: ему назначается тип, уровень уверенности и время обработки. Для пакетного режима далее возникает еще одно состояние - физическое размещение файла в новом каталоге внутри выходного архива.

Явное выделение состояний важно для проектирования БД и API. Каждое состояние порождает собственный набор данных для хранения и отображения, а переходы между состояниями определяют состав полей в summary.csv и таблицах batch_runs и batch_items.

3.7 Описание структуры базы данных

Для хранения эксплуатационной информации выбрана встроенная СУБД SQLite. Такой выбор позволяет избежать дополнительного серверного окружения и сделать проект полностью переносимым: база данных располагается внутри каталога приложения и инициализируется автоматически при первом запуске.

Структура БД включает пять сущностей. Пара classification_requests и classification_results описывает одиночные прогнозы по текстам и изображениям. Таблица model_registry хранит сведения о версиях моделей и достигнутых метриках. Для пакетной обработки выделены сущности batch_runs и batch_items, позволяющие восстановить состав архива, результат по каждому файлу и путь к сформированному выходному архиву.

Использование локальной базы данных решает сразу две задачи. Во-первых, обеспечивается воспроизводимость эксперимента и возможность повторного анализа результатов. Во-вторых, интерфейс получает доступ к истории запусков без ручного хранения журналов в текстовых файлах.

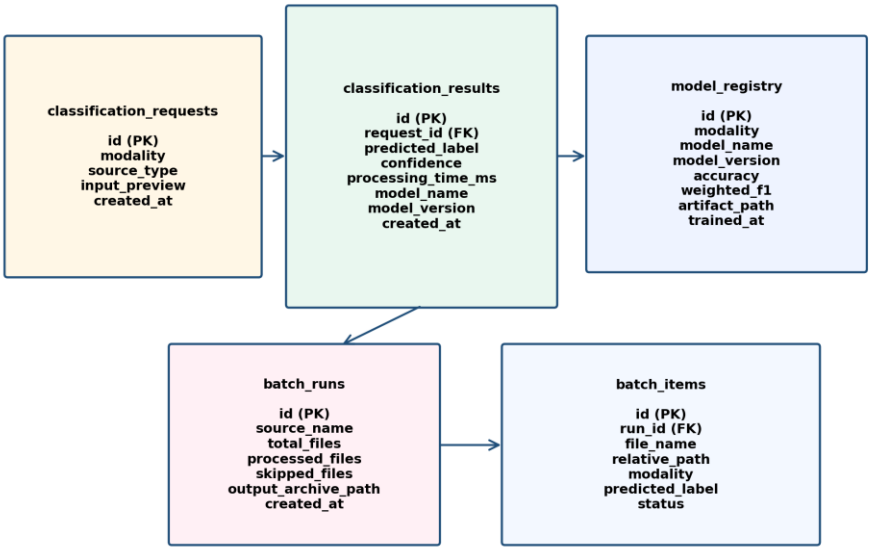

Рисунок 2.3 - Структура базы данных приложения

Таблица 2.3 - Таблицы и сущности базы данных

Таблица	**Назначение**
classification_requests	Фиксирует тип и источник одиночного запроса
classification_results	Хранит прогноз, уверенность, время ответа и сведения о модели
model_registry	Содержит реестр обученных моделей и их метрики
batch_runs	Описывает пакетный запуск по ZIP-архиву
batch_items	Фиксирует результат по каждому файлу внутри архива

3.8 Алгоритм обработки документов и архивов

Алгоритм работы системы начинается с загрузки пользователем исходных данных. Если передан текст, сервис сразу передает его в текстовую модель. Если пользователь загружает скан, изображение нормализуется и преобразуется в признаковый вектор фиксированного размера.

При пакетной обработке архив сначала безопасно распаковывается во временный каталог. Далее для каждого файла определяется модальность на

основе расширения. Текстовые документы анализируются через извлечение содержимого, а изображения направляются в визуальную модель. Неподдерживаемые форматы пропускаются, но также фиксируются в сводном отчете с указанием причины пропуска.

После получения прогнозов система создает новый каталог, где файлы раскладываются по папкам 'Договор', 'Счет', 'Приказ', 'Служебная записка' и 'Отчет'. Итоговый каталог упаковывается в ZIP-архив, путь к нему заносится в историю пакетных запусков, а пользователь получает готовый результат для дальнейшей передачи в делопроизводство.

С инженерной точки зрения алгоритм пакетной обработки интересен тем, что он связывает файловую подсистему, алгоритмы машинного обучения и хранилище истории в единую транзакционную последовательность. Ошибка на любом из этапов не должна приводить к потере уже обработанных файлов и должна фиксироваться в итоговой сводке.

Рисунок 2.4 - Алгоритм обработки документа или архива

3.9 Проектирование формата выходных данных

Одним из требований к приложению является удобство передачи результата в дальнейший контур документооборота. Пользователь не должен вручную копировать найденные файлы или выписывать список прогнозов после пакетного запуска.

По этой причине результат архивной обработки формируется в двух видах. Первый вид - это summary.csv, содержащий сведения по каждому файлу: исходный путь в архиве, модальность, распознанный класс и статус обработки. Второй вид - ZIP-архив, в котором документы уже физически разложены по папкам в соответствии с прогнозом системы.

Такая схема делает результат одновременно машиночитаемым и удобным для пользователя. CSV пригоден для контроля качества и последующего анализа, а ZIP-архив можно сразу передавать сотруднику или загружать в следующую систему.

Таблица 2.4 - Структура результатов пакетной сортировки

Элемент результата	**Назначение**
sorted_documents/Договор	Файлы, распознанные как договоры
sorted_documents/Счет	Файлы, распознанные как счета
sorted_documents/Приказ	Файлы, распознанные как приказы
sorted_documents/Служебная записка	Файлы, распознанные как служебные записки
sorted_documents/Отчет	Файлы, распознанные как отчеты
summary.csv	Сводка по всем файлам архива со статусом обработки

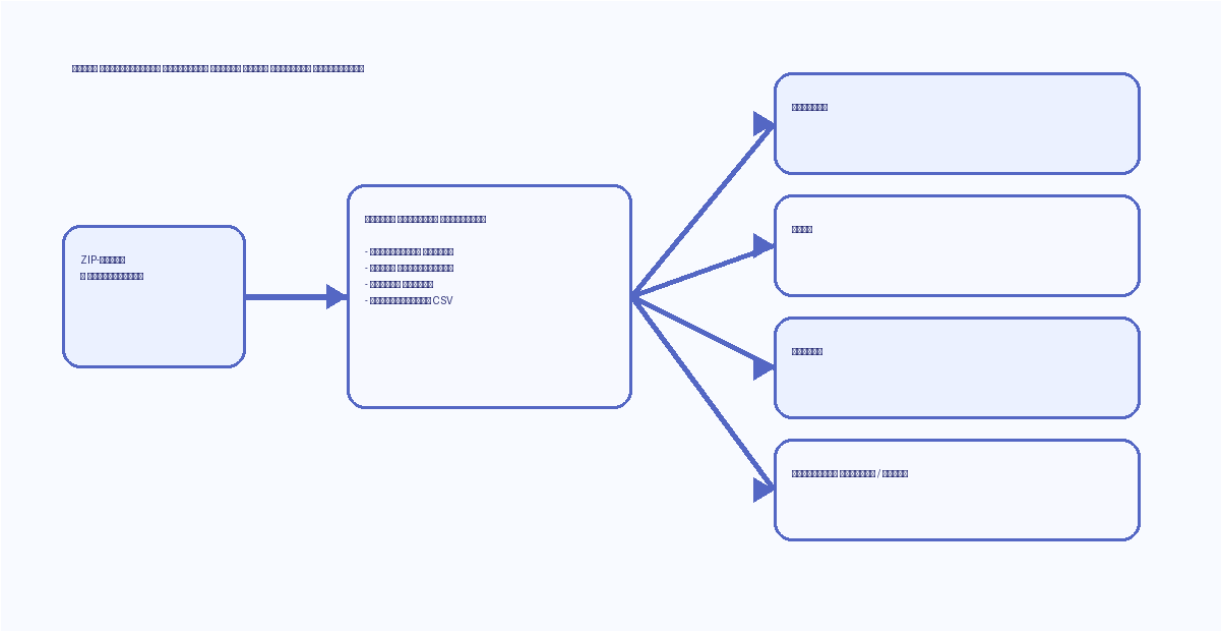

Рисунок 2.6 - Формирование итогового архива после пакетной сортировки

3.10 Взаимодействие интерфейса, API и прикладных модулей

Коммуникация между интерфейсом и серверной частью осуществляется через внутренние вызовы FastAPI. Такой подход обеспечивает единый путь обработки как для web-интерфейса, так и для автотестов, использующих HTTP-маршруты.

При одиночной классификации пользователь получает компактный ответ: тип документа, уверенность модели, время обработки и распределение вероятностей по классам. В пакетном режиме вместо одного прогноза возвращается сводка по архиву, таблица результатов и файл с готовым выходным архивом.

Сервисный слой изолирует UI от деталей реализации моделей и БД. Это решение важно с инженерной точки зрения: изменение алгоритма классификации или структуры базы данных не требует полной переработки интерфейса, пока сохраняется контракт методов сервисного слоя.

Рисунок 2.5 - Схема взаимодействия основных компонентов

3.11 Обоснование выбора локальной архитектуры

Для ВКР можно было бы спроектировать распределенную архитектуру с отдельным frontend, сервером приложений, внешней СУБД и объектным хранилищем. Однако такой подход усложнил бы разворачивание и сместил бы

акцент работы с прикладной задачи классификации на инфраструктурную интеграцию.

Локальная архитектура позволяет сосредоточиться именно на интеллектуальной составляющей и на прикладной ценности решения. Все ключевые элементы - модели, БД, интерфейс и пакетный модуль - находятся внутри одного репозитория и могут быть проверены без сетевой инфраструктуры и дополнительных сервисных зависимостей.

Кроме того, такой подход соответствует сценарию защиты и передачи проекта: приложение можно запустить на другой машине, воспроизвести результаты и продемонстрировать пакетную сортировку документов без предварительной административной подготовки окружения.

3.12 Выбор технологий и инструментальных средств

Для реализации приложения выбран стек Python 3.12, FastAPI, Gradio, scikit-learn, SQLite и python-docx. Указанные технологии достаточно зрелы, хорошо документированы и позволяют собрать прикладной сервис без избыточной инфраструктурной сложности.

FastAPI обеспечивает серверные маршруты, схемы запросов и прозрачное тестирование API. Gradio используется как low-code средство создания web-интерфейса: он позволяет сосредоточиться на логике обработки документов, а не на ручной frontend-разработке. SQLite поддерживает локальный режим работы и хорошо подходит для демонстрационного и исследовательского проекта.

Выбор классических алгоритмов машинного обучения объясняется требованиями к воспроизводимости и скорости. Логистическая регрессия на TF-IDF признаках интерпретируема для текстовой задачи, а Random Forest устойчиво работает на синтетических визуальных макетах документов и не требует длительного обучения на GPU.

Таблица 2.5 - Используемые технологии

Технология	**Роль в проекте**

Python 3.12	Единая среда разработки, обучения моделей и автоматизации
FastAPI	REST API, схемы запросов и сервисные маршруты
Gradio	Low-code web-интерфейс для демонстрации и работы пользователя
scikit-learn	Построение текстовой и визуальной моделей классификации
SQLite	Локальное хранение истории и метаданных моделей
python-docx	Извлечение текста из DOCX при пакетной обработке
pytest	Автоматическое тестирование критических сценариев

3.13 Выводы по второй главе

Во второй главе спроектирована архитектура программной системы, определены пользовательские сценарии, структура базы данных и состав технологического стека. Особое внимание уделено пакетному сценарию обработки архива как ключевому носителю практической ценности работы.

Построенная архитектура обеспечивает модульность, переносимость и возможность дальнейшего наращивания функциональности. На следующем этапе проектирования эти решения переходят в конкретную программную реализацию.

4 ГЛАВА 3 РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ И ПОЭТАПНАЯ РЕАЛИЗАЦИЯ БЛОКОВ СИСТЕМЫ

4.1 Общая организация проекта

Практическая реализация выполнена в формате самостоятельного Python-проекта, пригодного для открытия в среде PyCharm. Внутри репозитория выделены каталоги project и docs: первый содержит исходный код, модели и тесты, второй включает пояснительную записку, презентацию и сопроводительные материалы.

Исходный код разделен на функциональные модули: конфигурация, генерация синтетических данных, обучение моделей, обработка архивов, сервисный слой, API, UI и тесты. Такое разбиение снижает связность компонентов и облегчает расширение проекта, например добавление новых типов документов или альтернативных моделей.

Кроме собственно приложения, в проект включены скрипты автоматической сборки артефактов и выпускных материалов. За счет этого тезисы пояснительной записки привязаны к фактически полученным метрикам, диаграммам и скриншотам, а не являются статическим описанием, оторванным от рабочего кода.

Таблица 3.1 - Основные программные модули проекта

Модуль	**Назначение**
config.py	Конфигурация путей, классов, параметров моделей и артефактов
data/text_samples.py	Генерация корпуса текстов для пяти типов документов
data/image_generator.py	Синтез макетов сканов документов и подготовка признаков
models/*	Обучение и сериализация текстовой и графической моделей
batch_processing.py	Разбор ZIP, извлечение файлов, упаковка результата
service.py	Бизнес-логика классификации, журналирование, пакетная обработка

api.py	Публикация REST-интерфейса для текста, изображений и архивов
ui.py	Low-code интерфейс Gradio с тремя прикладными сценариями

4.2 Этап 1. Подготовка целевых классов и синтетических данных

Первым этапом разработки стала формализация прикладных классов документов и определение их характерных признаков. Для текстовой классификации были выделены устойчивые лексические шаблоны, связанные с договорами, счетами, приказами, служебными записками и отчетами.

На основе этих шаблонов был сформирован программно генерируемый корпус текстовых документов. Такой подход позволяет контролировать состав классов, изменять количество примеров и воспроизводимо получать набор данных без зависимости от внешних архивов, содержащих конфиденциальную информацию.

Для визуальной классификации реализован генератор макетов документов. Он строит синтетические сканы страниц: размещает блоки текста, таблицы, подписи и диаграммы, добавляет шум, размытие и затемнение. В результате получается контролируемая учебная выборка, имитирующая разные типы офисных документов.

4.3 Детализация подготовки текстового корпуса

При генерации текстового корпуса особое значение имело не простое копирование нескольких шаблонов, а вариативность формулировок внутри каждого класса. Для этого были выделены заголовочные конструкции, основные смысловые блоки, уточняющие детали и завершающие формулы, которые затем комбинировались между собой.

Такой подход позволяет сформировать большой набор осмысленных предложений без ручного составления каждой строки. Одновременно он

сохраняет контроль над семантическими границами классов и снижает риск того, что модель выучит случайные, а не предметно значимые слова.

Дополнительно корпус проектировался так, чтобы тексты оставались похожими на реальные деловые документы. Это важно для демонстрации: пользователь должен видеть, что приложение классифицирует не искусственные метки, а документоподобные формулировки, близкие к реальной деловой переписке.

4.4 Этап 2. Реализация текстового классификатора

Текстовый модуль реализован на основе пайплайна scikit-learn. Сначала текст преобразуется в TF-IDF признаки с учетом униграмм и биграмм, после чего вектор признаков передается в модель логистической регрессии. Такой подход хорошо подходит для задач тематической и жанровой классификации документов малого и среднего объема.

При разработке модуля было важно добиться не только высокого качества, но и короткого времени ответа. Логистическая регрессия обеспечивает быстрый инференс, а вероятностный вывод позволяет возвращать в интерфейс распределение по всем классам, что повышает интерпретируемость результата.

Обученная текстовая модель сериализуется в файл artifacts/models/text_classifier.joblib и затем подгружается при запуске приложения. Это позволяет отделить фазу обучения от эксплуатационного режима и сократить время старта сервиса на пользовательской машине.

4.5 Этап 3. Реализация классификатора сканов документов

Второй интеллектуальный блок ориентирован на распознавание типа документа по структуре его визуального макета. Каждое изображение переводится в оттенки серого, приводится к размеру 48x64 пикселя и разворачивается в признаковый вектор. Такое представление позволяет использовать классический алгоритм без сложной нейросетевой инфраструктуры.

В качестве классификатора выбран Random Forest. Он устойчив к шуму в данных, хорошо работает на синтетических выборках и позволяет различать характерные макеты документов: счет с табличной областью, приказ с нумерованными пунктами, служебную записку с кратким телом и отчет с графическими элементами.

Подобный подход не претендует на замену современных transformer-моделей обработки документов, однако в рамках ВКР он оправдан с инженерной точки зрения: модель легко обучается локально, демонстрирует воспроизводимый результат и хорошо объясняет связь между визуальной структурой документа и его типом.

4.6 Детализация визуального генератора документов

Генератор визуальных данных строится не вокруг абстрактных геометрических фигур, а вокруг структурных паттернов документов. Для счета формируется крупная табличная область и блок итогов, для договора - плотные абзацы и парные подписи, для приказа - нумерованные пункты, для отчета - сочетание таблицы и диаграммы.

После построения макета к изображению добавляются шум, небольшое размытие и градиент затемнения, что имитирует типичные артефакты сканирования и фотографирования. Эти искажения важны для того, чтобы модель не зависела исключительно от идеального синтетического рисунка страницы.

В результате формируется компромисс между реалистичностью и контролируемостью набора данных. Для учебного проекта такой подход предпочтителен, поскольку позволяет показать полный цикл построения визуального классификатора без использования внешних закрытых или слишком объемных датасетов.

Рисунок 3.1 - Примеры синтетических макетов документов для визуальной модели

4.7 Этап 4. Реализация подсистемы пакетной сортировки архива

Наиболее значимый с прикладной точки зрения этап разработки связан с пакетной сортировкой ZIP-архива. Именно этот блок превращает проект из демонстратора одиночной классификации в сервис предварительной обработки массива документов.

Подсистема архивной обработки выполняет безопасную распаковку, определяет поддерживаемые форматы, извлекает текст из TXT, MD и DOCX, а изображения направляет в визуальную модель. Для каждого файла формируется запись со статусом обработки, классом, уверенностью и служебным комментарием.

После завершения обработки создается новая файловая структура sorted_documents/<тип документа>/..., затем она упаковывается в ZIP-архив и возвращается пользователю. Дополнительно сохраняется summary.csv, который

может быть использован для последующей проверки качества маршрутизации и анализа состава архива.

В процессе реализации отдельное внимание уделялось поддержке нескольких модальностей внутри одного пакета. Это означает, что в одном архиве могут присутствовать как текстовые файлы, так и изображения, и для каждого из них система должна корректно выбрать свой алгоритмический маршрут.

Таблица 3.3 - Структура выходного архива пакетной сортировки

Элемент результата	**Назначение**
sorted_documents/Договор	Файлы, распознанные как договоры
sorted_documents/Счет	Файлы, распознанные как счета
sorted_documents/Приказ	Файлы, распознанные как приказы
sorted_documents/Служебная записка	Файлы, распознанные как служебные записки
sorted_documents/Отчет	Файлы, распознанные как отчеты
summary.csv	Сводка по всем файлам архива со статусом обработки

4.8 Пооперационный маршрут пакетного запуска

Пооперационно пакетный запуск состоит из нескольких шагов. Сначала пользователь передает ZIP-архив и сервис создает временный рабочий каталог. Затем архив извлекается с контролем путей и проверкой, что ни один элемент не выходит за границы допустимой папки извлечения.

Далее выполняется проход по всем файлам. Для каждого файла определяется модальность и выбирается способ обработки: чтение текста, извлечение содержимого из DOCX либо подготовка изображения для визуальной модели. После получения прогноза файл копируется в каталог соответствующего класса внутри структуры результата.

На завершающем шаге формируются summary.csv, ZIP-архив результата и запись в истории пакетных запусков. Именно такая детализация делает пакетный

режим не вспомогательной функцией, а полноценным прикладным блоком с собственной логикой, журналированием и пользовательской ценностью.

4.9 Этап 5. Реализация слоя хранения и журналирования

Для обеспечения воспроизводимости эксперимента и наглядности интерфейса в проект встроено локальное журналирование. Одиночные запросы и пакетные прогоны фиксируются в SQLite с сохранением времени обработки, источника данных и достигнутых метрик модели.

Отдельного внимания заслуживает реестр моделей. При пересборке артефактов система не накапливает устаревшие абсолютные пути, а заменяет записи в model_registry актуальными относительными путями внутри проекта. Это было важно для переносимости приложения на другие машины и предотвращения ошибок запуска.

Наличие полноценного слоя хранения повышает практическую ценность решения: пользователь может не только получить текущий прогноз, но и просматривать историю эксплуатации, проверять результаты пакетной сортировки и контролировать версии применяемых моделей.

4.10 Этап 6. Реализация REST API

API-слой построен на FastAPI и выступает единым внешним контрактом приложения. Через него реализованы маршруты проверки готовности сервиса, получения истории и метрик, а также классификации текста, изображений и ZIP-архивов.

Использование pydantic-схем повышает надежность работы API: входные данные валидируются, а ответы имеют прогнозируемую структуру. Для файловых маршрутов реализована дополнительная проверка форматов, что важно для устойчивой работы пакетного режима.

API используется не только интерфейсом, но и тестами. Благодаря этому критические пользовательские сценарии проверяются на том же уровне абстракции, который видит внешний потребитель приложения.

Таблица 3.2 - Основные маршруты REST API

Маршрут	**Назначение**
GET /api/health	Проверка готовности сервиса и моделей
GET /api/models	Получение списка моделей и метрик
GET /api/history	Получение истории одиночных запросов
GET /api/batch/history	Получение истории пакетных прогонов
POST /api/text/classify	Классификация текста документа
POST /api/image/classify	Классификация скана документа
POST /api/batch/classify-archive	Обработка ZIP-архива документов

4.11 Организация демонстрационных данных и сценариев показа

Для защиты ВКР важно не только наличие кода, но и наличие удобного сценария демонстрации. Поэтому проект автоматически создает демонстрационные сканы документов и демонстрационный ZIP-архив, в котором собраны текстовые и визуальные примеры всех пяти классов.

Такой набор данных решает сразу две задачи. Во-первых, он делает проверку приложения воспроизводимой: на любой машине можно запустить один и тот же архив и получить понятный результат. Во-вторых, он позволяет показывать пакетный сценарий без использования внешних конфиденциальных документов.

Наличие демонстрационного архива также упростило создание автотестов. Пакетный режим проверяется не на абстрактных моках, а на реальном ZIP-файле, структура которого воспроизводится средствами самого проекта.

4.12 Этап 7. Реализация low-code интерфейса

Интерфейс приложения реализован на Gradio и встроен в серверное приложение по адресу /ui. На главной странице выделены три вкладки, соответствующие прикладным сценариям: классификация текста, классификация скана и пакетная сортировка архива.

Для каждой вкладки предусмотрены отдельные элементы ввода и отдельные формы визуализации результата. Одиночные сценарии возвращают тип документа и распределение вероятностей по классам. Пакетный сценарий возвращает сводную статистику, таблицу по всем файлам и ссылку на сформированный архив результата.

Правая часть интерфейса используется как информационная панель. В ней отображаются метрики обученных моделей, история одиночных запросов и история пакетных прогонов. Такое решение делает интерфейс не только рабочим инструментом, но и стендом для демонстрации инженерных результатов проекта.

Так как интерфейс напрямую связан с API и сервисным слоем, его корректность дополнительно подтверждается автотестами маршрутов и ручной функциональной проверкой в рамках главы о тестировании. Для документации и защиты основной акцент сделан на диаграммах взаимодействия и итоговых таблицах результатов.

4.13 Этап 8. Обеспечение переносимости и запуска на другой машине

Во время разработки отдельной задачей стала переносимость проекта между рабочими машинами. Практика показала, что при перемещении репозитория могут появляться ошибки из-за абсолютных путей в базе данных, артефактах или скриптах запуска. Поэтому в проекте были устранены machine-specific ссылки и оставлены только относительные пути внутри репозитория.

Дополнительно были добавлены корневые точки входа main.py и generate_assets.py, чтобы пользователь мог запускать приложение и пересборку артефактов из корня репозитория без ручного перехода по каталогам. Это упростило как демонстрацию, так и разворачивание проекта на другом компьютере.

Отдельно доработана логика выбора порта: если порт 8000 уже занят, приложение автоматически подбирает следующий свободный локальный порт.

Такое решение уменьшает количество ошибок запуска при защите, тестировании и переносе проекта на новую рабочую станцию.

4.14 Этап 9. Автоматизация сборки артефактов и выпускных материалов

В проект включены специальные сценарии пересборки моделей, графиков, диаграмм, пояснительной записки и презентации. Такая автоматизация важна для поддержания согласованности между кодом, метриками, скриншотами и текстом ВКР.

Генерация артефактов запускается отдельным скриптом. Если модели уже соответствуют текущей конфигурации, повторное обучение не выполняется, что ускоряет старт приложения и исключает лишнюю пересборку данных. При изменении состава классов или версии модели артефакты пересоздаются автоматически.

В результате проект получает свойства инженерно оформленного продукта: пользователь может открыть репозиторий, пересобрать модели, прогнать тесты, пересоздать документацию и получить одинаковый результат на другой машине.

4.15 Выводы по третьей главе

В третьей главе поэтапно описана реализация всех ключевых блоков системы: от генерации данных и обучения моделей до пакетной сортировки архива, REST API, интерфейса и автоматизации сборки артефактов.

Полученная реализация демонстрирует, что даже компактное локальное приложение может обладать выраженной практической полезностью, если результаты классификации непосредственно встроены в прикладной сценарий работы с массивом документов.

5 ГЛАВА 4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ, АНАЛИЗ РЕЗУЛЬТАТОВ И ОЦЕНКА РЕШЕНИЯ

5.1 Организация тестирования и критерии оценки

Цель тестирования заключалась в проверке работоспособности всех уровней приложения: одиночной классификации текста и сканов, пакетной сортировки архива, API-маршрутов, слоя хранения и пользовательского интерфейса.

Для оценки качества моделей использовались метрики accuracy, precision, recall и F1-score. Дополнительно оценивались время ответа после прогрева приложения, корректность формирования выходного архива и сохранение истории обработки в локальной базе данных.

Среднее время обработки текстового документа составило 0 мс, медианное значение - 0.0 мс. Для скана документа среднее время обработки составило 41.43 мс, медианное - 40.0 мс. Такие значения подтверждают пригодность приложения для интерактивной эксплуатации.

Таблица 4.2 - Показатели производительности после прогрева

Сценарий	**Среднее время, мс**	**Медианное время, мс**
Текстовая классификация	0.00	0.00
Классификация скана	41.43	40.00

5.2 Автоматическое тестирование программного продукта

Для автоматической проверки проекта реализован набор из шестнадцати тестов на базе pytest. Он покрывает сервисный слой, обработку файлов по пути, генерацию артефактов, API-маршруты, пакетную сортировку архива и стартовую логику приложения.

Покрытие исходного кода по результатам запуска pytest --cov превышает 85%, что позволяет считать критические сценарии использования приложения

надежно зафиксированными тестами. Особенно важным является тестирование пакетного режима, поскольку он затрагивает файловую систему, архивирование и множественные вызовы моделей.

Таблица 4.1 - Основные сценарии автоматического тестирования

№	**Компонент**	**Проверка**	**Ожидаемый результат**
1	text service	Распознавание счета по текстовому описанию	Класс 'Счет'
2	image service	Распознавание приказа по синтетическому скану	Класс 'Приказ'
3	image service	Классификация изображения по файловому пути	Класс 'Договор'
4	training pipeline	Проверка генерации моделей, отчетов и диаграмм	Все артефакты существуют
5	registry	Контроль путей артефактов в model_registry	Только относительные пути внутри проекта
6	batch service	Пакетная обработка ZIP-архива	Создается итоговый архив и сводка
7	startup	Повторный ensure_ready не перезагружает модели	Инициализация выполняется однократно
8	text API	Маршрут POST /api/text/classify	Корректный JSON-ответ
9	image API	Маршрут POST /api/image/classify	Корректный JSON-ответ
10	batch API	Маршрут POST /api/batch/classify-archive	Возвращается сводка по архиву

5.3 Результаты оценки качества моделей

После обучения на синтетических выборках обе модели показали высокие значения точности. Это объясняется тем, что набор данных создавался с контролируемыми и отчетливо различимыми признаками, характерными для выбранных типов документов.

Высокие значения accuracy и F1-score в рамках данной работы следует трактовать как подтверждение корректности спроектированной архитектуры и стабильности программной реализации, а не как окончательную оценку качества на реальном промышленном корпусе документов. Для промышленного внедрения потребовалась бы валидация на живых данных.

Таблица 4.3 - Итоговые метрики качества моделей

Модель	**Accuracy**	**Precision**	**Recall**	**F1-score**
Текстовая модель	1.0000	1.0000	1.0000	1.0000
Модель изображений	1.0000	1.0000	1.0000	1.0000

Таблица 4.4 - Показатели качества текстовой модели по классам

Класс	**Precision**	**Recall**	**F1-score**	**Поддержка**
Договор	1.0000	1.0000	1.0000	40
Отчет	1.0000	1.0000	1.0000	40
Приказ	1.0000	1.0000	1.0000	40
Служебная записка	1.0000	1.0000	1.0000	40
Счет	1.0000	1.0000	1.0000	40

Таблица 4.5 - Показатели качества модели сканов по классам

Класс	**Precision**	**Recall**	**F1-score**	**Поддержка**
Договор	1.0000	1.0000	1.0000	60
Отчет	1.0000	1.0000	1.0000	60
Приказ	1.0000	1.0000	1.0000	60
Служебная записка	1.0000	1.0000	1.0000	60

Счет	1.0000	1.0000	1.0000	60

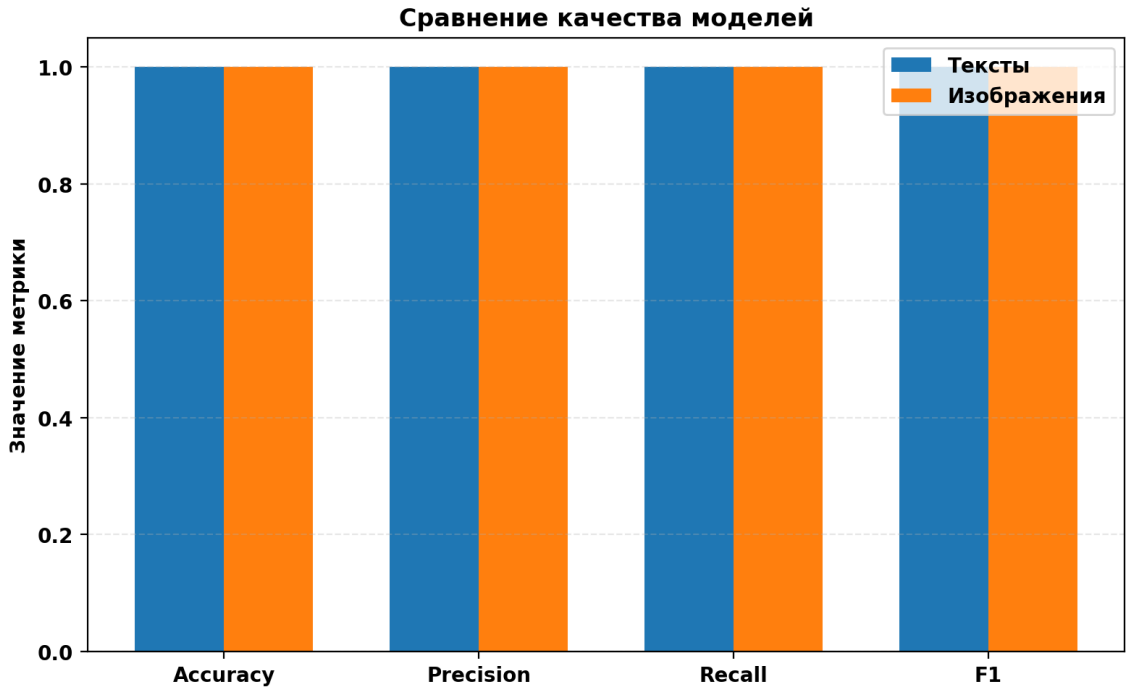

Рисунок 4.1 - Сравнение итоговых метрик качества моделей

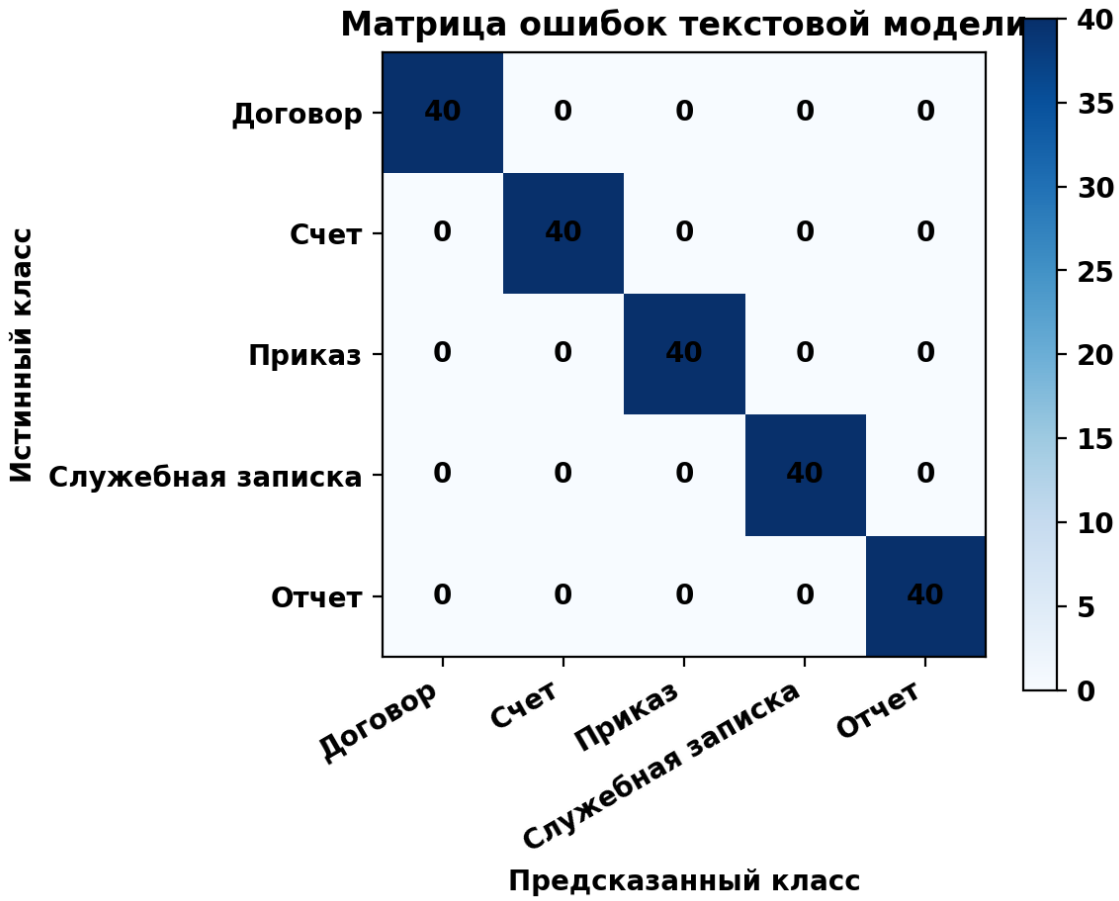

Рисунок 4.2 - Матрица ошибок текстовой модели

Рисунок 4.3 - Матрица ошибок модели сканов

5.4 Результаты пакетной сортировки демонстрационного архива

Для оценки прикладного сценария был использован демонстрационный ZIP-архив, автоматически сформированный из пяти текстовых и пяти визуальных документов. Архив содержит по два экземпляра каждого класса: текстовый и графический вариант документа.

Пакетная обработка завершилась созданием итогового архива с разложением файлов по каталогам. Таким образом, приложение подтвердило способность не только классифицировать отдельные документы, но и решать прикладную задачу расфасовки смешанного архива по типам документов.

Таблица 4.6 - Распределение файлов по типам документов в демо-архиве

Тип документа	Количество файлов
Договор	2
Отчет	2
Приказ	2
Служебная записка	2
Счет	2

Таблица 4.7 - Фрагмент сводки по файлам демо-архива

Файл	Путь в архиве	Модальность	Класс	Статус
договор.png	договор.png	image	Договор	processed
договор.txt	договор.txt	text	Договор	processed
отчет.png	отчет.png	image	Отчет	processed
отчет.txt	отчет.txt	text	Отчет	processed
приказ.png	приказ.png	image	Приказ	processed
приказ.txt	приказ.txt	text	Приказ	processed
служебная_записка.png	служебная_записка.png	image	Служебная записка	processed
служебная_записка.txt	служебная_записка.txt	text	Служебная записка	processed
счет.png	счет.png	image	Счет	processed
счет.txt	счет.txt	text	Счет	processed

5.5 Оценка производительности и устойчивости

Даже при наличии трех разных пользовательских сценариев система остается отзывчивой в локальном режиме. Это достигается за счет предварительной инициализации моделей, повторного использования

артефактов и отказа от лишней пересборки данных, если конфигурация проекта не менялась.

Пакетный режим устойчив к неоднородному составу архива. Неподдерживаемые файлы не приводят к аварийному завершению процесса, а помечаются как пропущенные с комментарием. Такой подход важен для реального делопроизводства, где входящие архивы могут содержать служебные файлы, превью или вложения непредусмотренных форматов.

С точки зрения надежности дополнительно важна корректная работа после переноса проекта на другую машину. Использование относительных путей к артефактам, локальной базы данных и автоподбора порта существенно снижает риск ошибки запуска и делает решение пригодным для повторной демонстрации в новой среде.

5.6 Интерпретация результатов по классам документов

Высокие метрики по каждому классу показывают, что выбранные признаки действительно отражают структуру документов. Для текстовой модели решающую роль играют устойчивые деловые формулировки, характерные для договоров, счетов, приказов, служебных записок и отчетов.

Для визуальной модели различимость достигается за счет макетных особенностей: наличие или отсутствие таблиц, компоновка абзацев, зона подписей и графические элементы отчета. Такой результат подтверждает гипотезу о том, что даже без OCR структура страницы уже содержит значимую информацию о типе документа.

Следовательно, в рамках ВКР достигнут не только программный, но и методический результат: показано, что прикладная задача маршрутизации документов может решаться как по семантическим, так и по визуальным признакам, а объединение этих подходов внутри одного приложения повышает его практическую выразительность.

5.7 Преимущества и недостатки разработанного решения

К основным преимуществам приложения относятся локальный режим работы, отсутствие внешней СУБД, наличие low-code интерфейса, воспроизводимая генерация артефактов и практический пакетный сценарий обработки архивов. Пользователь получает законченный цикл от загрузки документа до формирования нового отсортированного архива.

С инженерной точки зрения сильной стороной решения является согласованность компонентов. Код, тесты, модели, диаграммы, таблицы метрик, пояснительная записка и презентация формируются из одного репозитория, что упрощает сопровождение и защиту проекта.

К недостаткам текущей версии следует отнести использование синтетических обучающих данных и ограниченный набор поддерживаемых типов документов. Кроме того, модель визуальной классификации пока не анализирует содержимое документа на уровне OCR, а опирается только на геометрию макета.

Перспективным направлением развития является подключение реальных корпусов документов, расширение перечня типов файлов, добавление OCR и использование более сложных мультимодальных моделей. Однако для целей ВКР текущая реализация уже демонстрирует законченный прикладной эффект и инженерную состоятельность.

Для опытной эксплуатации в организации система могла бы использоваться как предобработчик входящей корреспонденции или как вспомогательный инструмент сотрудника делопроизводства. Даже если окончательное решение по документу принимает человек, предварительная сортировка по типам уже экономит время и снижает вероятность ошибок маршрутизации.

5.8 Оценка сценариев практического применения

Первым возможным сценарием применения является предварительный разбор входящего почтового архива подразделения. Сотрудник может выгрузить полученные за день файлы, запустить пакетную сортировку и сразу получить разложение по типам документов, пригодное для дальнейшей ручной проверки и распределения.

Второй сценарий связан с учебными и исследовательскими лабораториями. Приложение можно использовать как демонстрационный стенд, показывающий полный контур прикладного ML-проекта: генерацию данных, обучение моделей, публикацию API, пакетную обработку архива и контроль качества через тесты.

Третий сценарий состоит в использовании проекта как заготовки для последующего расширения. На его основе можно подключить OCR, реальные корпоративные данные, дополнительные типы документов и более сложные модели, не разрушая уже реализованную архитектуру и накопленный слой тестов.

5.9 Выводы по четвертой главе

Тестирование подтвердило работоспособность разработанного программного продукта. Автоматические тесты проверяют критические сценарии, а итоговые таблицы и графики демонстрируют высокое качество моделей на сформированных выборках и корректную работу пакетного режима.

Полученные результаты позволяют считать цель ВКР достигнутой: приложение решает практическую задачу классификации и первичной сортировки документов, обладает воспроизводимой архитектурой и готово к демонстрации.

6 Заключение

В ходе выполнения выпускной квалификационной работы была разработана программная система интеллектуальной классификации и пакетной сортировки документов на языке Python с web-интерфейсом, реализованным на low-code платформе Gradio, и серверной частью на FastAPI.

В работе была уточнена практическая постановка задачи: вместо абстрактной демонстрации мультимодальной классификации сформирован прикладной сценарий первичной маршрутизации документов. Приложение умеет определять тип отдельного документа по тексту и по скану, а также обрабатывать ZIP-архив и раскладывать файлы по каталогам в соответствии с их типами.

В первой главе проанализирована предметная область документооборота, обоснована актуальность темы и проведено сравнение с существующими решениями. Во второй главе спроектированы архитектура приложения, структура базы данных и пользовательские сценарии. В третьей главе реализованы программные блоки, обеспечивающие обучение моделей, обработку архивов, работу API и интерфейса. В четвертой главе выполнено тестирование и анализ результатов.

Практический результат работы представляет собой готовое локальное приложение, которое запускается без внешней инфраструктуры, проходит автоматическое тестирование, хранит историю обработки и формирует комплект отчетных материалов. Разработанная система обладает выраженной практической ценностью и может служить как демонстрационным стендом, так и основой для дальнейшего развития в сторону промышленной обработки документов.

7 Список использованных источников

1. Python Software Foundation. Python documentation [Электронный ресурс]. URL: https://docs.python.org/3/ (дата обращения: 24.04.2026).

2. Ramalho L. Fluent Python. 2nd ed. Sebastopol: O'Reilly Media, 2022. 1014 p.

3. Géron A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. 3rd ed. Sebastopol: O'Reilly Media, 2022. 851 p.

4. FastAPI Documentation [Электронный ресурс]. URL: https://fastapi.tiangolo.com/ (дата обращения: 24.04.2026).

5. Gradio Documentation [Электронный ресурс]. URL: https://www.gradio.app/docs (дата обращения: 24.04.2026).

6. Scikit-learn User Guide [Электронный ресурс]. URL: https://scikit-learn.org/stable/user_guide.html (дата обращения: 24.04.2026).

7. SQLite Documentation [Электронный ресурс]. URL: https://sqlite.org/docs.html (дата обращения: 24.04.2026).

8. python-docx Documentation [Электронный ресурс]. URL: https://python-docx.readthedocs.io/ (дата обращения: 24.04.2026).

9. pytest Documentation [Электронный ресурс]. URL: https://docs.pytest.org/en/stable/ (дата обращения: 24.04.2026).

10. Goodfellow I., Bengio Y., Courville A. Deep Learning. Cambridge: MIT Press, 2016. 800 p.

11. Bishop C. M. Pattern Recognition and Machine Learning. New York: Springer, 2006. 738 p.

12. ABBYY FlexiCapture [Электронный ресурс]. URL: https://www.abbyy.com/flexicapture/ (дата обращения: 24.04.2026).

13. Directum RX. Управление документами [Электронный ресурс]. URL: https://www.directum.ru/solution/rx_document_management (дата обращения: 24.04.2026).

14. ELMA365 ECM [Электронный ресурс]. URL: https://elma365.com/ru/products/ecm/enterprise-content-management/ (дата обращения: 24.04.2026).

15. 1С:Документооборот [Электронный ресурс]. URL: https://v8.1c.ru/doc8/ (дата обращения: 24.04.2026).

8 Приложение А

8.1 Структура проекта и инструкция по запуску

Проект состоит из прикладной части project и комплекта выпускных материалов docs. В папке project расположен исходный код, тесты, локальная база данных и артефакты моделей. В папке docs находятся пояснительная записка, презентация и вспомогательные текстовые материалы.

Для запуска приложения из корня проекта используется команда python main.py либо запуск файла project/main.py. Web-интерфейс становится доступным по адресу http://127.0.0.1:8000/ui или на следующем свободном порту, если порт 8000 уже занят.

Для пересборки моделей, графиков, диаграмм и демонстрационного архива используется команда python generate_assets.py. Для запуска автотестов применяется команда python -m pytest --cov=project/vkr_classifier --cov-report=term-missing project/tests.

Демонстрационный архив для пакетной сортировки формируется автоматически и располагается по пути artifacts/batch_exports/demo_document_archive.zip.

Таблица А.1 - Основные элементы структуры проекта

Элемент	**Назначение**
project/vkr_classifier	Исходный код приложения и ML-пайплайна
project/tests	Автотесты сервисного слоя, API и логики запуска
project/artifacts/models	Сериализованные модели классификации
project/artifacts/tables	CSV и JSON-отчеты с метриками
project/artifacts/figures	Диаграммы, матрицы ошибок и служебные схемы
project/artifacts/demo_examples	Демонстрационные сканы документов
project/artifacts/batch_exports	Демо-архив и результаты пакетной сортировки

docs	Пояснительная записка, презентация и сопроводительные материалы